

RR_Car_Price_Prediction_Report

2. Analysis and feature engineering

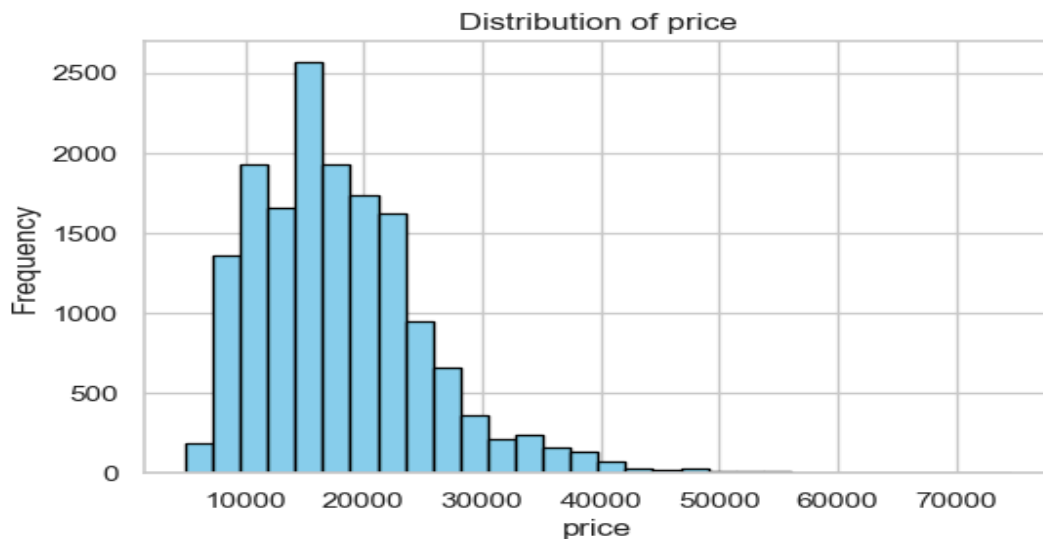
2.1 Preliminary analysis and frequency distributions

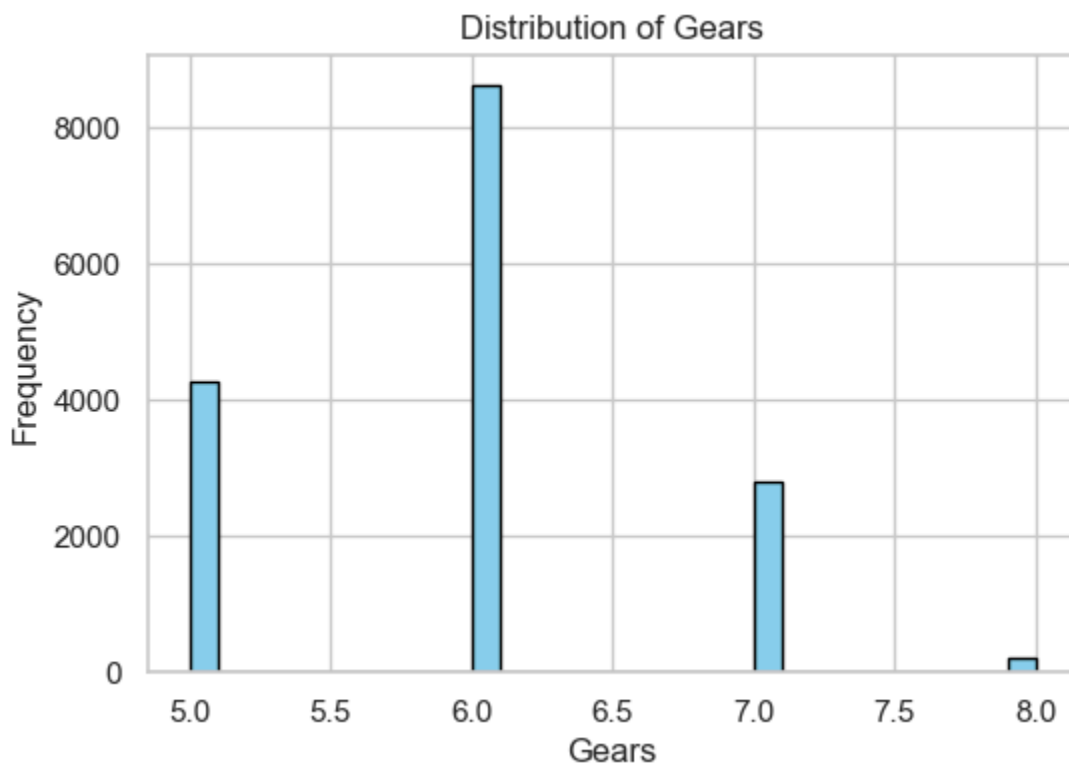
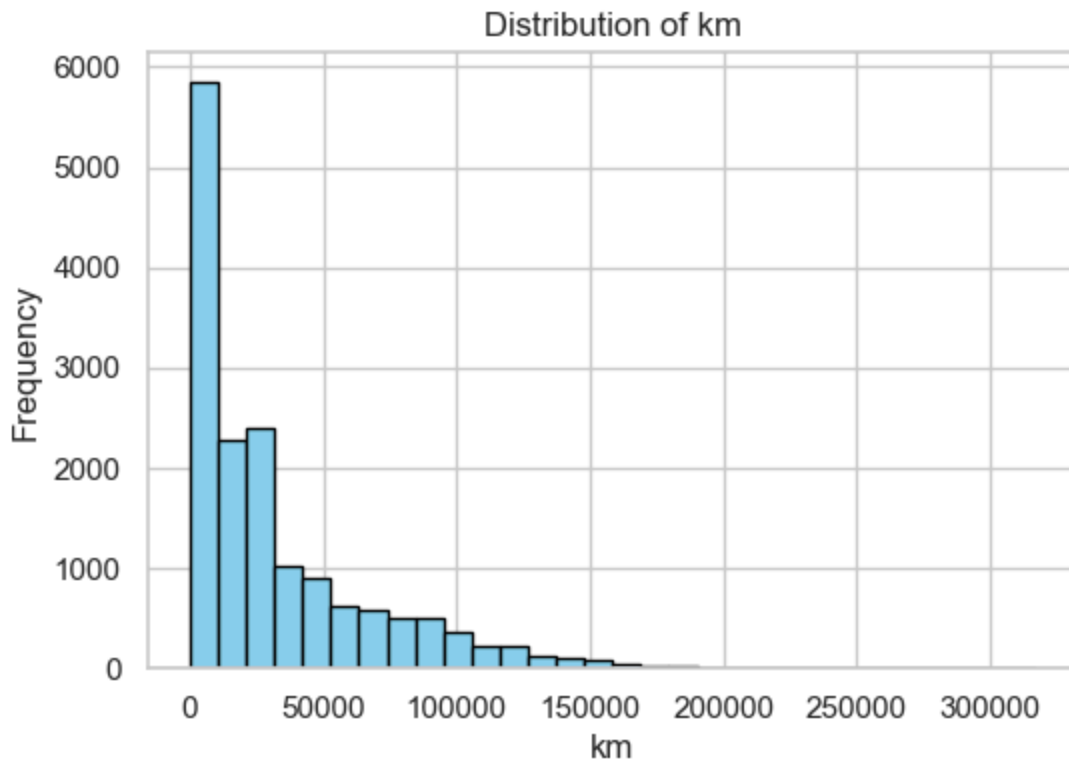
2.1.1 Find missing values

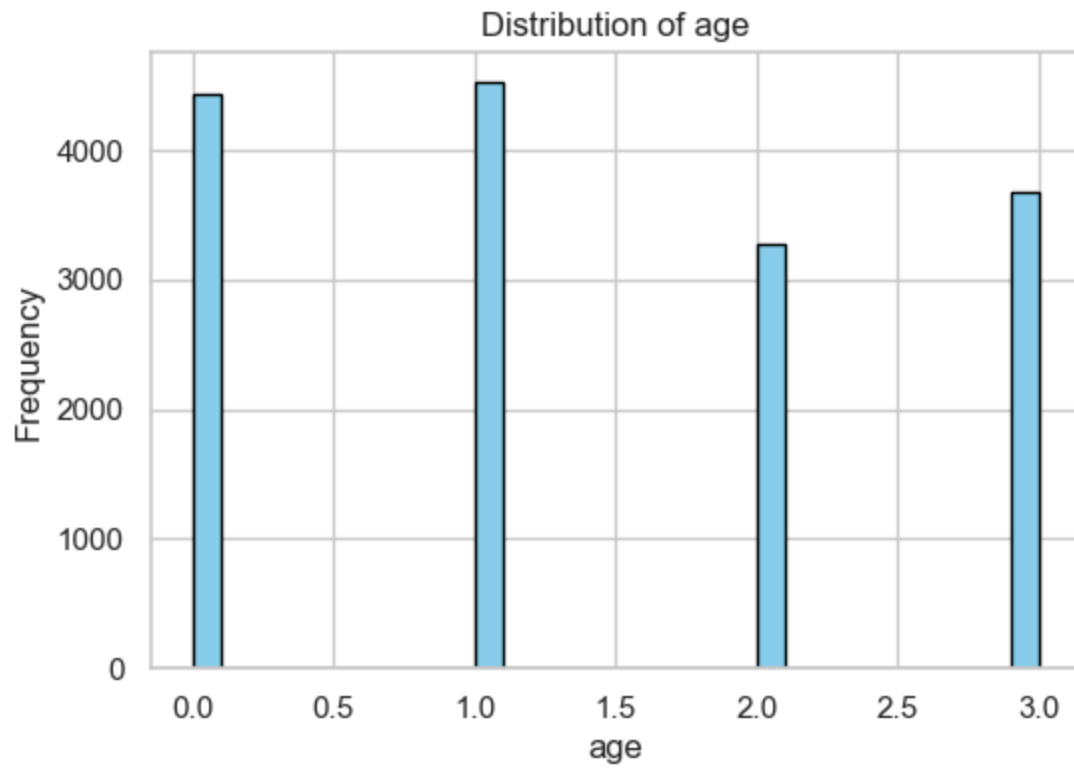
First, I calculated the percentage of missing data for every column. I dropped any columns that were missing more than 40% of their data because they do not have enough info to be useful. For the remaining columns, I filled the missing numerical values with the median and the missing text values with the mode. The dataset is now clean with zero missing values.

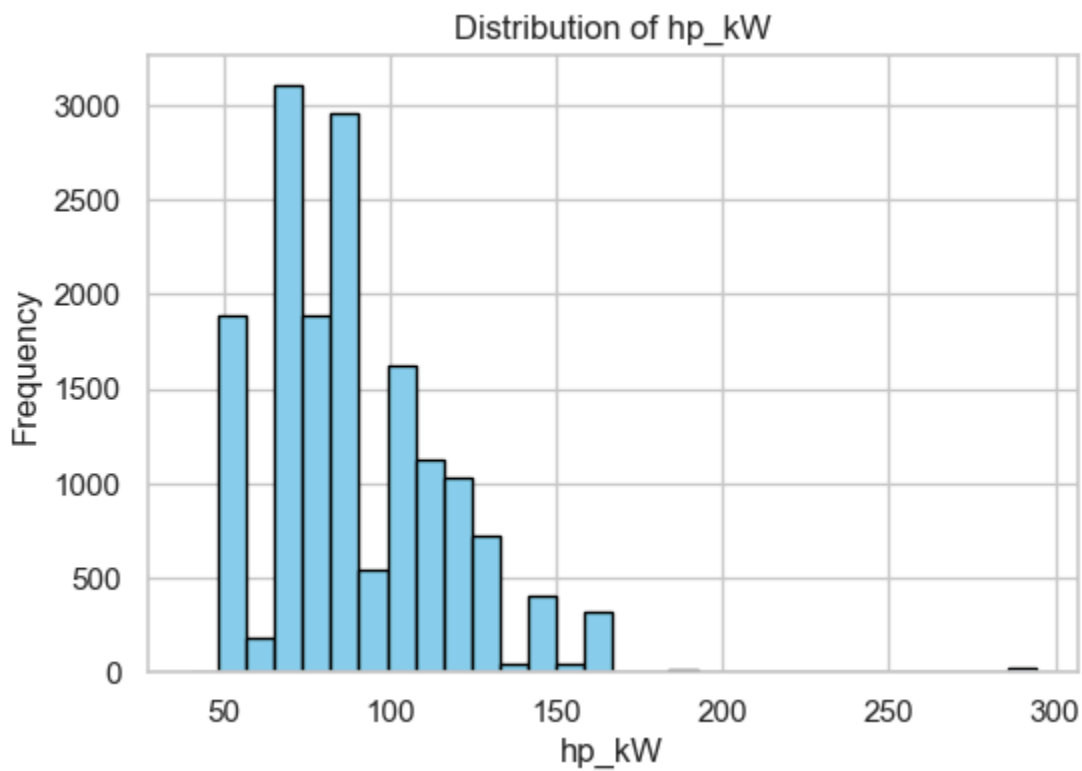
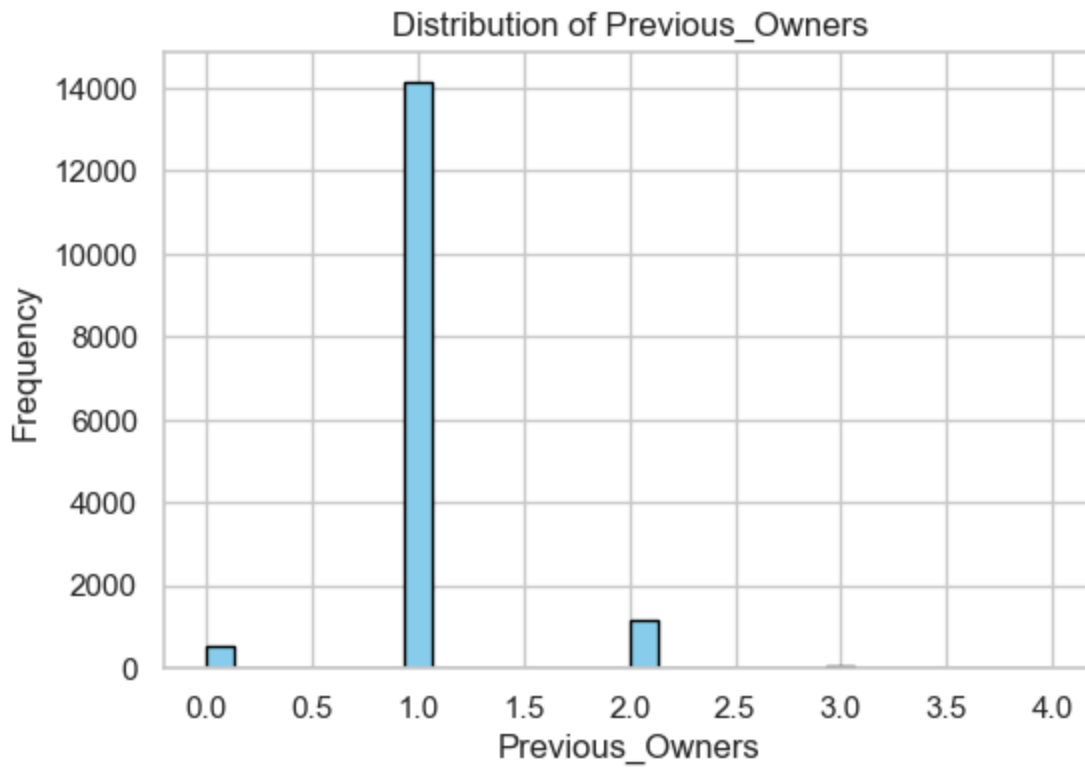
2.1.2 Identify numerical predictors and plot their frequency distributions

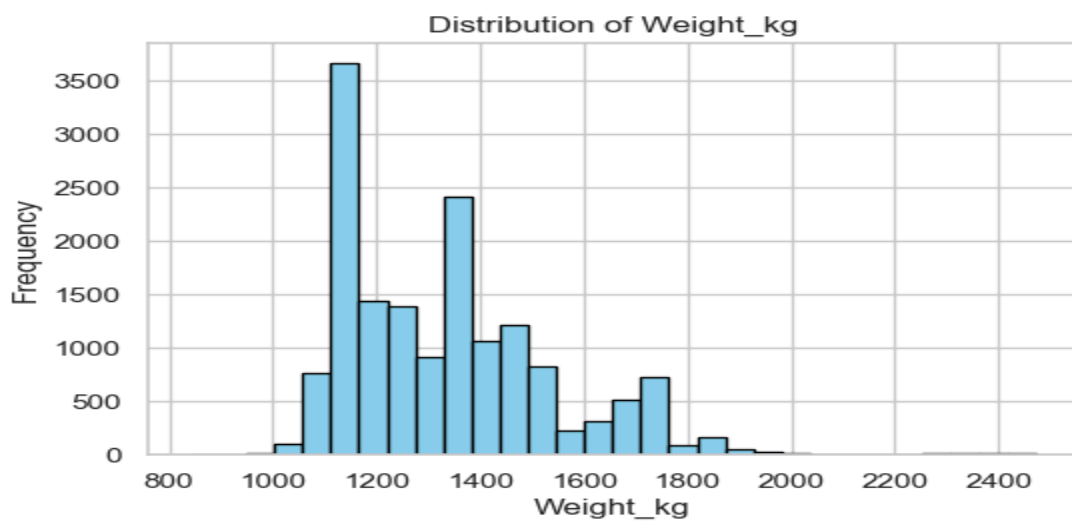
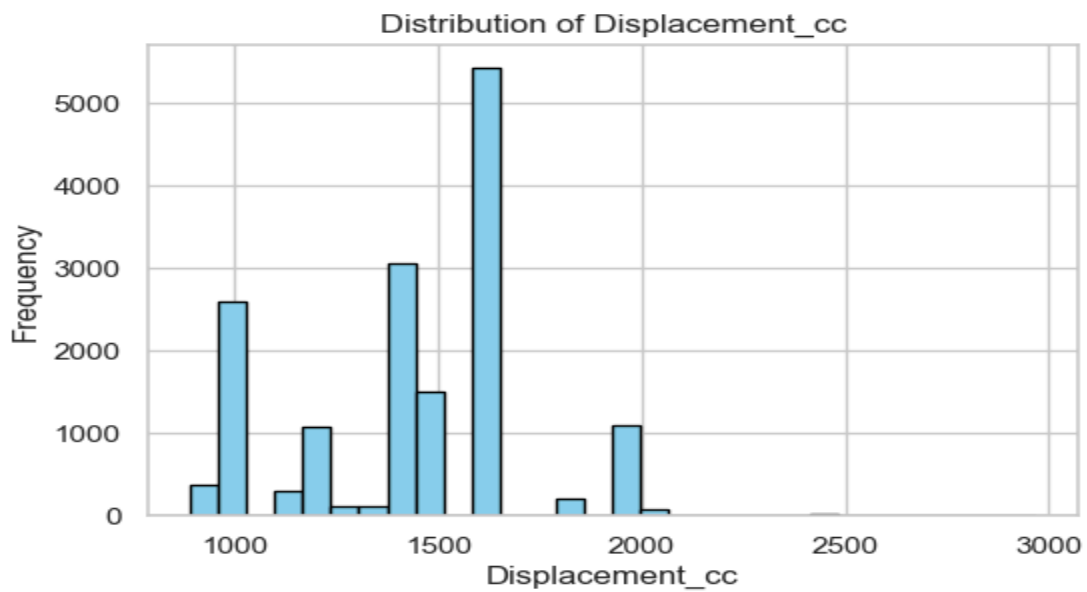
I identified the numerical columns in the dataset by filtering for integer and float data types. To understand how the data is spread out, I used a loop to create a histogram for each numerical predictor. Looking at these plots helps visualize the frequency distribution of each feature and makes it easy to spot any skewness or extreme values in the data.

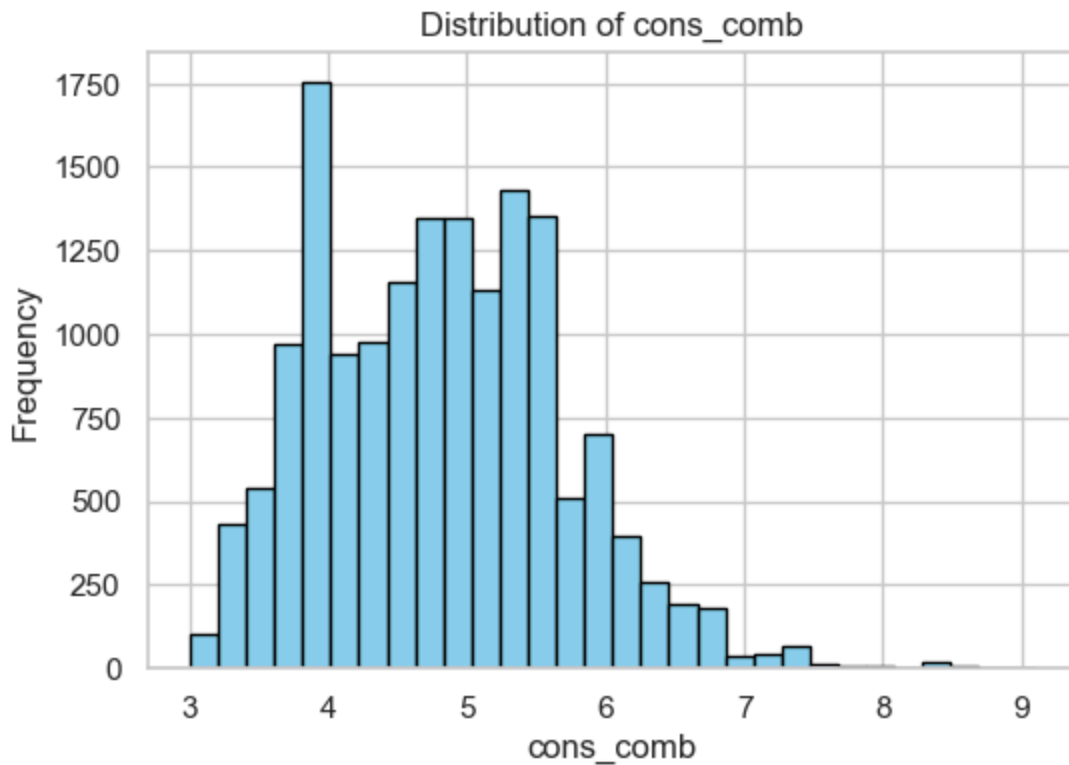






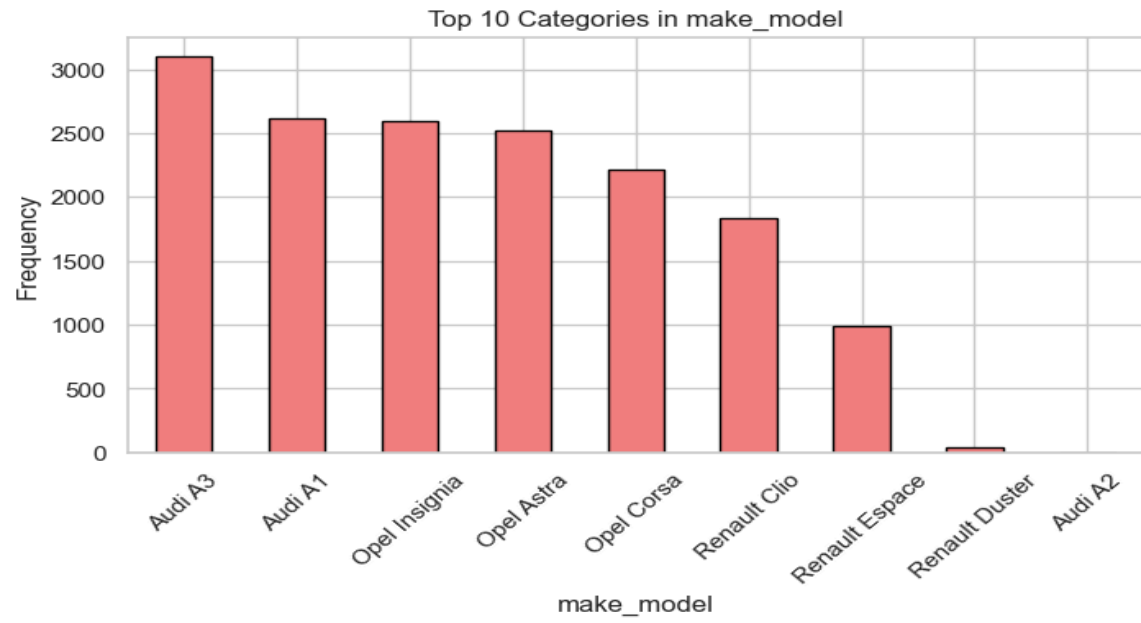


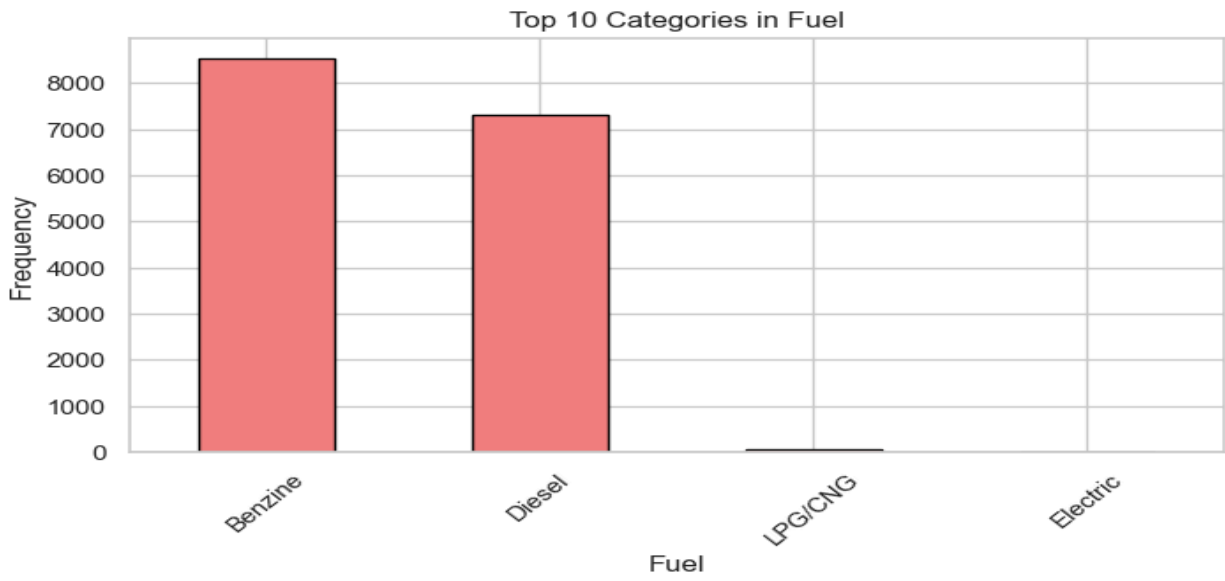
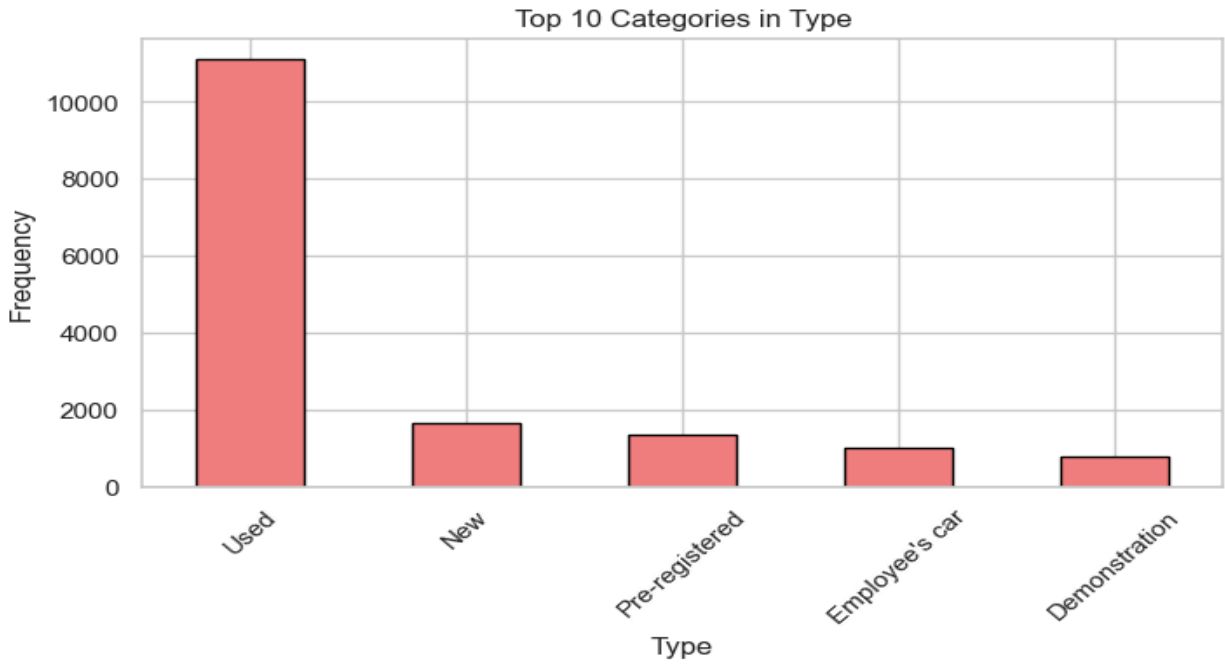


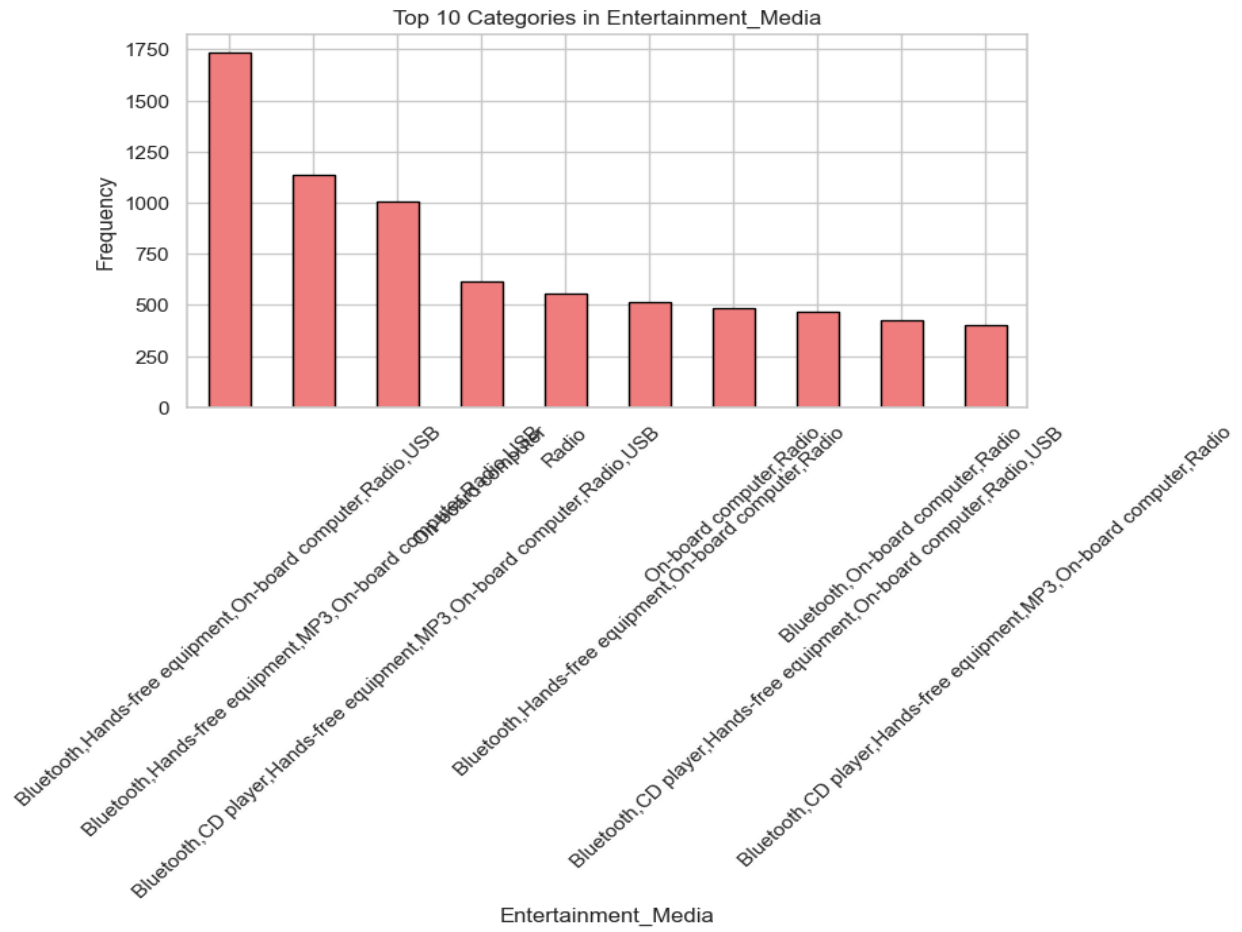


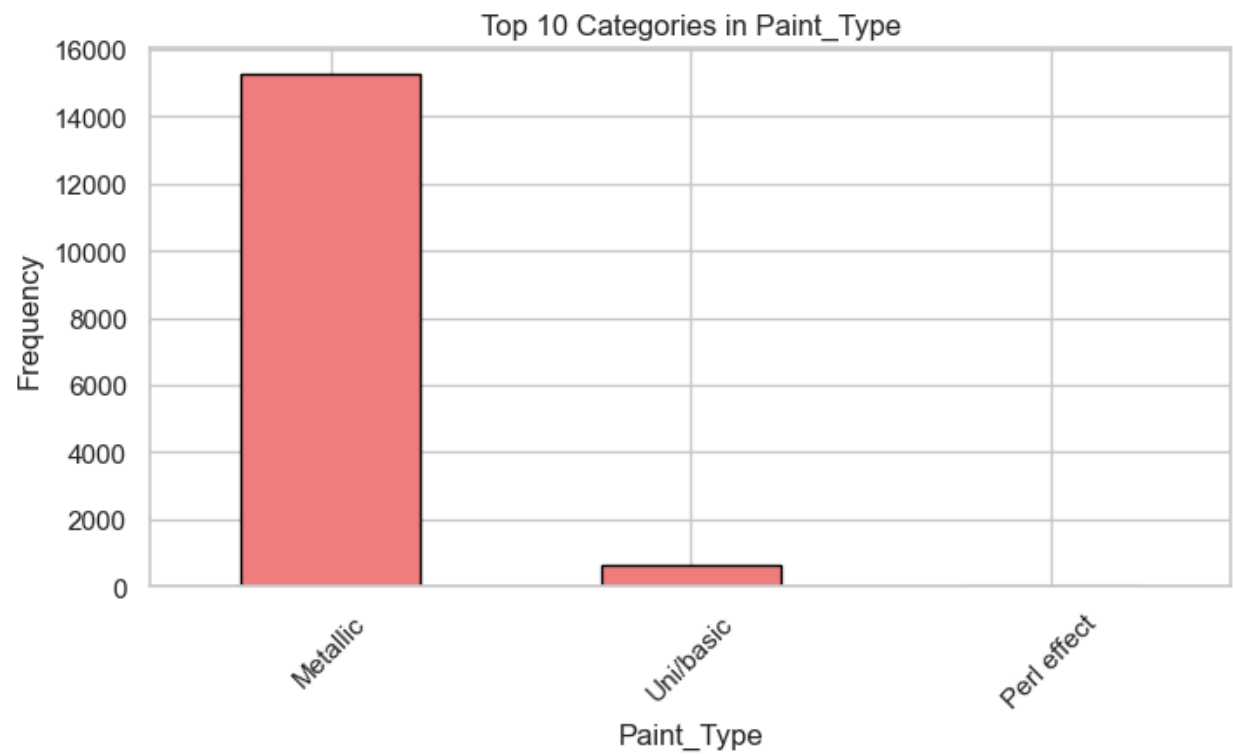
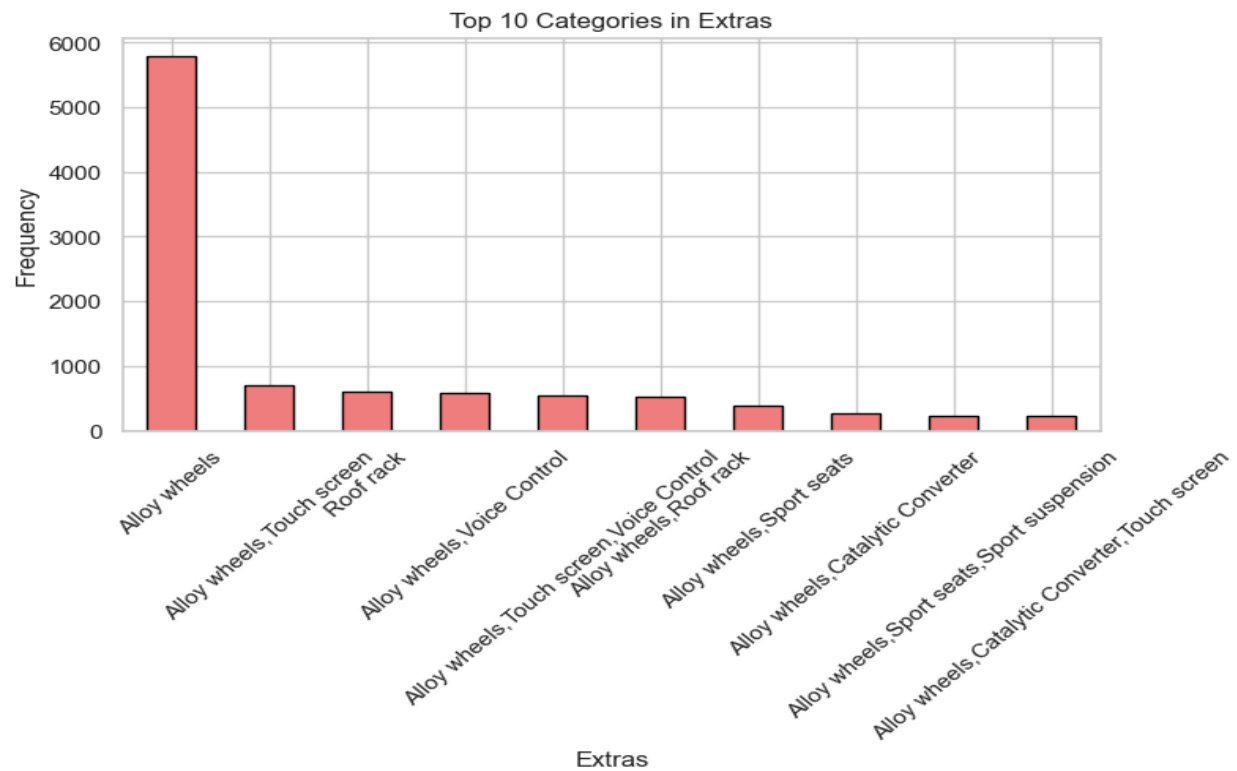
2.1.3 Identify categorical columns and check their frequency distributions

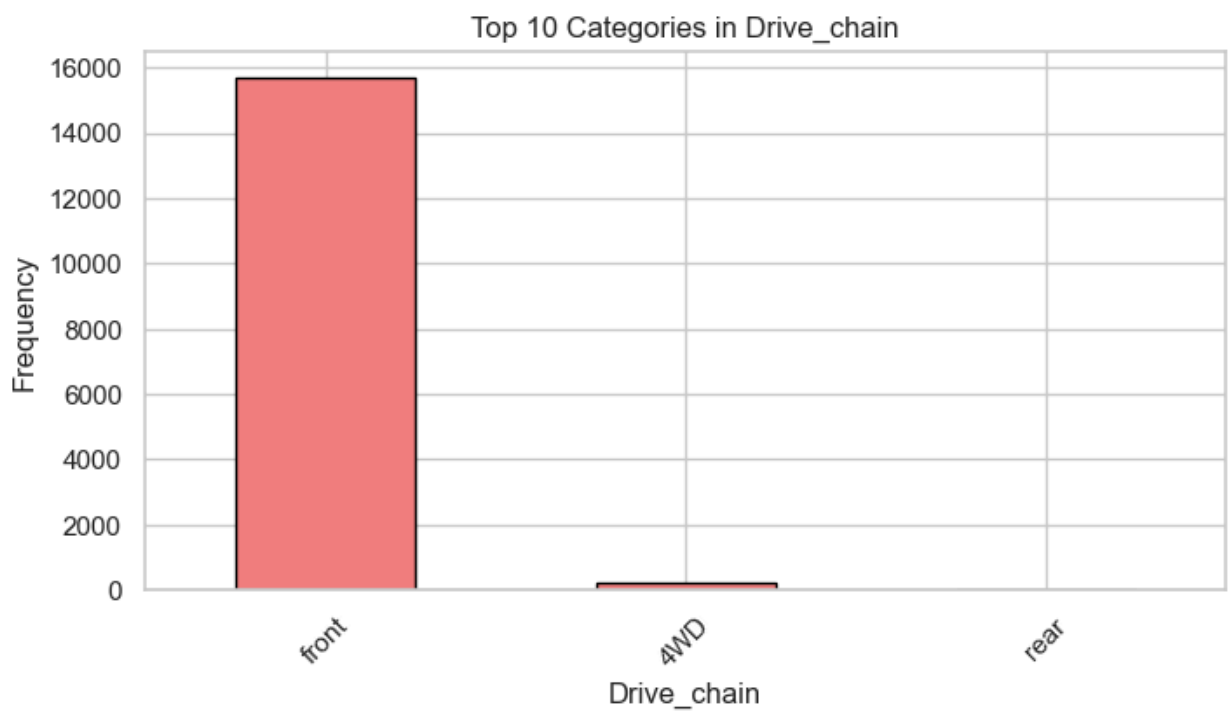
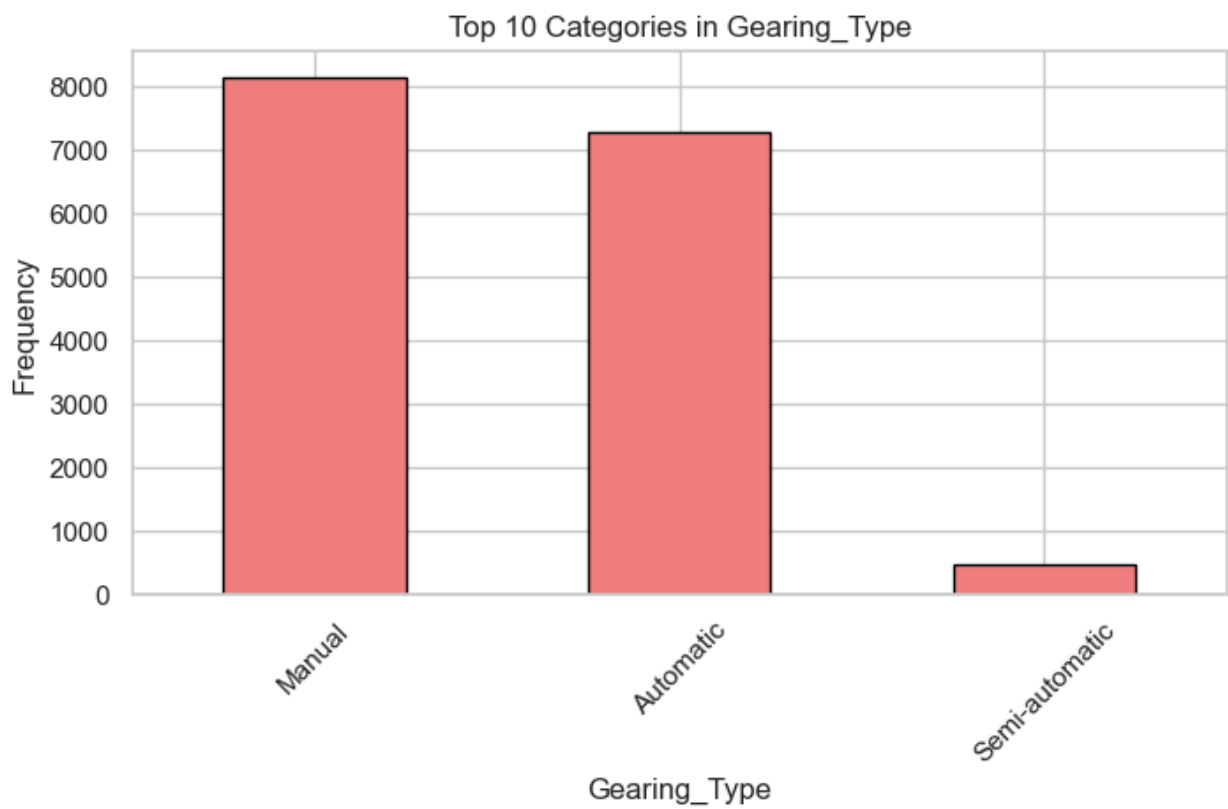
I identified all the categorical (text-based) columns in the dataset. To see how these groups were distributed, I used a loop to calculate the frequency counts and create bar charts for each categorical variable. Because some columns (like car models) have many unique text values, I limited the plots to show only the top 10 most frequent categories. This allowed me to easily see which categories are the most common and check for any major class imbalances.











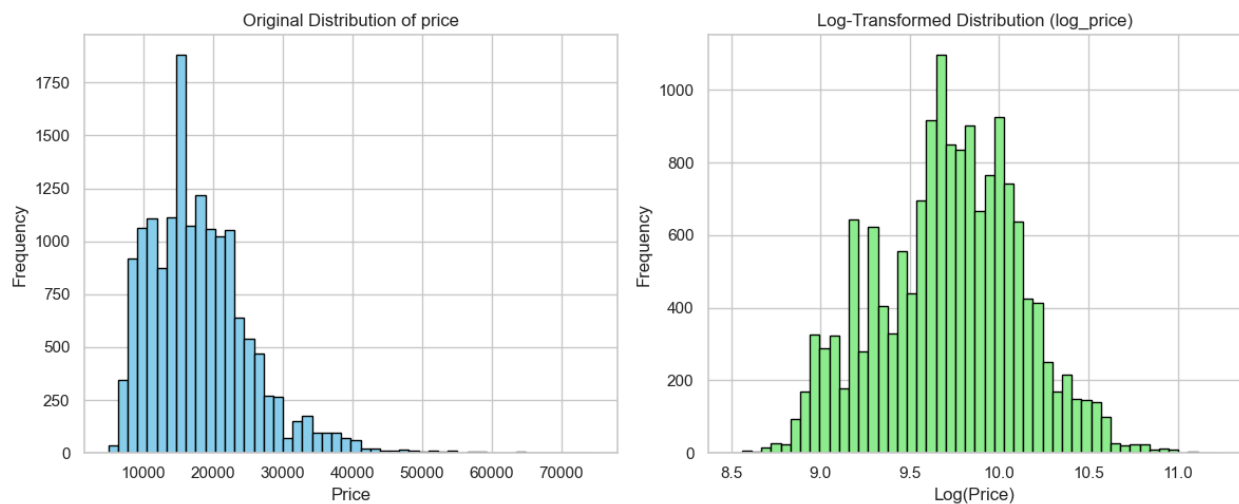
2.1.4 Fix columns with low frequency values and class imbalances.

To handle class imbalances, I first looked at the Type column. The categories "Pre-registered", "New", "Employee's car", and "Demonstration" all essentially represent cars with very low usage. To fix the low frequency of these individual labels, I grouped them all together into a single "Like New" category, which balances well against standard "Used" cars.

For the rest of the categorical columns, I applied a standard approach: if any specific category appeared in less than 1% of the dataset, I grouped it into a new "Other" category. This prevents the model from force-fitting to very rare text values (like a unusual car color) while keeping the most important info intact.

2.1.5 Identify the target variable and plot the frequency distributions. Apply necessary transformations.

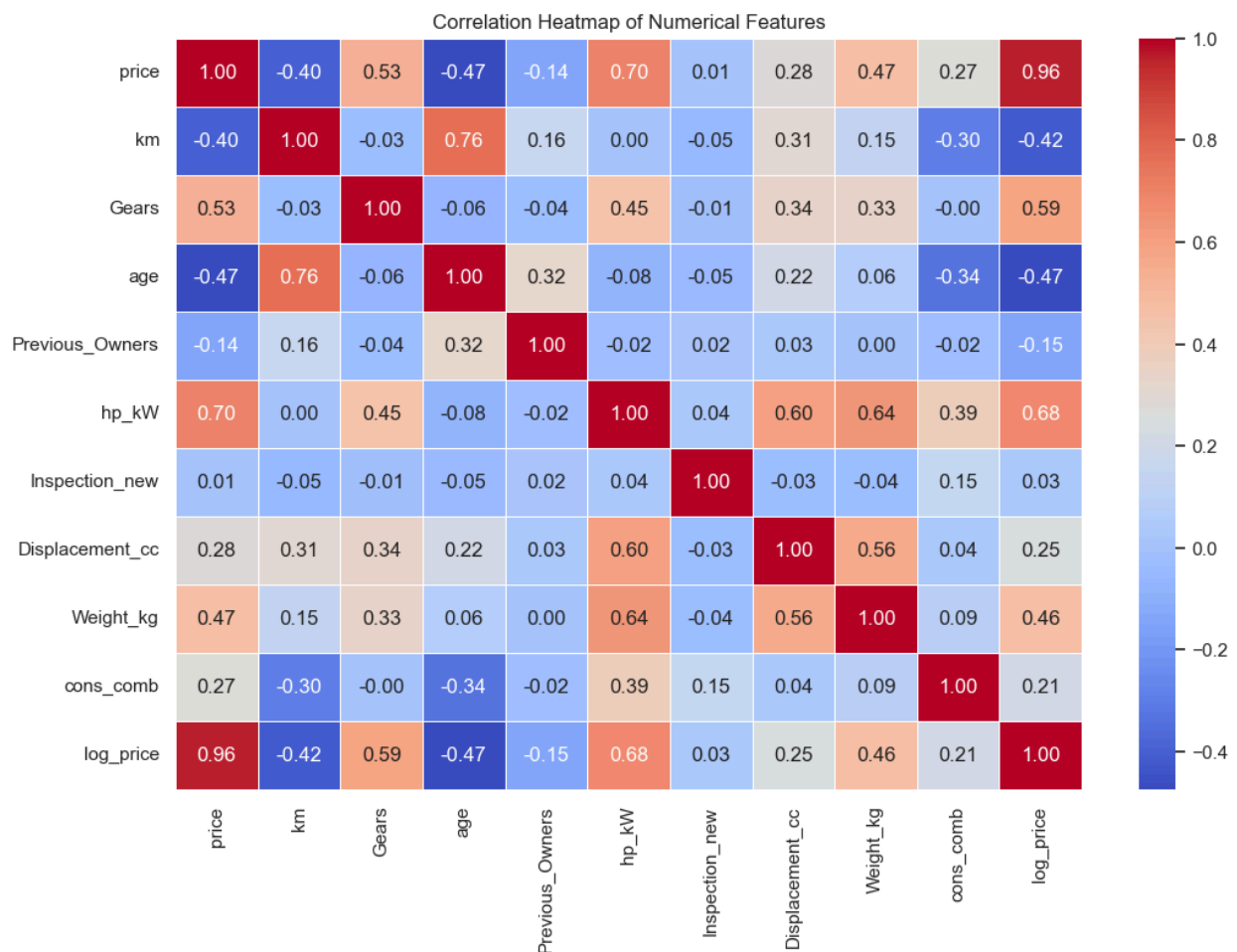
The target variable for this predictive model is the car's price. I plotted its original frequency distribution and observed that the data is highly right-skewed, as most cars fall into the lower-to-middle price brackets, with only a long tail of very expensive luxury vehicles. Because regularised linear regression models perform much better when the target variable is normally distributed, I applied a log transformation to the price. The resulting plot for the new log_price variable shows a much more symmetrical, bell-shaped distribution, which will help the model learn and predict more accurately.



2.2 Correlation Analysis

2.2.1 Plot the correlation map between features and the target variable.

To understand the relationships between the numerical features and the target variable, I generated a correlation matrix and visualised it using a heatmap. This allowed me to quickly see which variables move together. I highly focused on how each feature correlated with the car's price. Features showing a high positive or negative correlation with the price will act as the strongest predictors for the linear regression model, while variables that highly correlate with each other might need to be removed later to avoid multicollinearity.



2.2.2 Analyse correlation between categorical features and target variable.

To analyse the relationship between the categorical features and the target variable, I calculated and compared the average car price across the different text categories. By grouping the data and observing the mean price variations (for example, seeing how average prices fluctuate

significantly between different car makes or fuel types), I confirmed that these categorical features have a strong impact on the target variable.

2.3 Outlier Analysis

2.3.1 Identify potential outliers in the data.

To identify potential outliers in the dataset, I focused on the numerical columns and applied the standard Interquartile Range (IQR) method. By calculating the first (Q1) and third (Q3) quartiles for each feature, I established a normal data range (defined as 1.5 times the IQR below Q1 and above Q3). I then calculated the exact count of data points falling outside these boundaries for every numerical column. This approach provided a clear identification of which features contained extreme values that might require handling ahead.

2.3.2 Handle outliers

For the outliers, I didn't want to just delete the rows because we lose too much valuable data. Instead, I used capping. I basically took the IQR upper and lower bounds I calculated in the last step and clipped the extreme values to match those limits. So, if a number was way too high, I just capped it at the upper limit, and did the same for the bottom end. This way, the extreme values are fixed, but we still get to keep the exact same number of rows for the model.

2.4 Feature Engineering

2.4.1 Fix any redundant columns and create new ones if needed.

I checked the dataset and dropped any columns that only had one unique value, as they offer no predictive power. I also created a new `car_age` feature by subtracting the manufacturing year from the current year, and then dropped the original year column to avoid redundancy.

2.4.2 Analysis and feature engineering on `['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security']`.

First, I split the comma-separated lists in these columns into individual binary variables so the model could see exactly which features each car had. Next, I checked the frequency of each new feature and identified the ones that appeared in over 95% of cars or less than 5% of cars, as these don't explain much variance in price. Finally, I dropped those extremely common and rare features, along with the original text columns, to keep the dimension down.

2.4.3 Feature Encoding

To finish up feature engineering, I had to convert all the remaining text-based categorical columns into numbers so the linear regression model could process them. I used One-Hot Encoding to turn these categories into binary columns (0s and 1s). I also made sure to drop the first dummy column for each category to avoid the dummy variable trap, which would have caused multicollinearity issues for the model.

2.4.4 Split the data into training and testing sets.

Training set: 12732 rows

Testing set: 3183 rows

Before building the model, I needed to split the dataset into training and testing sets. I separated all my engineered features into the X variable and set my log_price as the y target variable. I then used a standard 80/20 split, meaning the model will use 80% of the data to learn the patterns, while the remaining 20% is kept completely hidden to test how accurately it can predict prices on unseen cars.

2.4.5 Scale the features

For the final feature engineering step, I applied a standard scaler to all the predictor variables. I made sure to fit the scaler only on the training data to avoid data leakage, and then transformed both the training and testing sets. This puts all the features on the same scale, ensuring that columns with naturally larger numbers (like mileage) don't overpower columns with smaller numbers (like car age) in the linear regression model.

3. Linear Regression Models

3.1 Baseline Linear Regression Model

3.1.1 Build and fit a basic linear regression model. Perform evaluation using suitable metrics.

Baseline Linear Regression Performance:

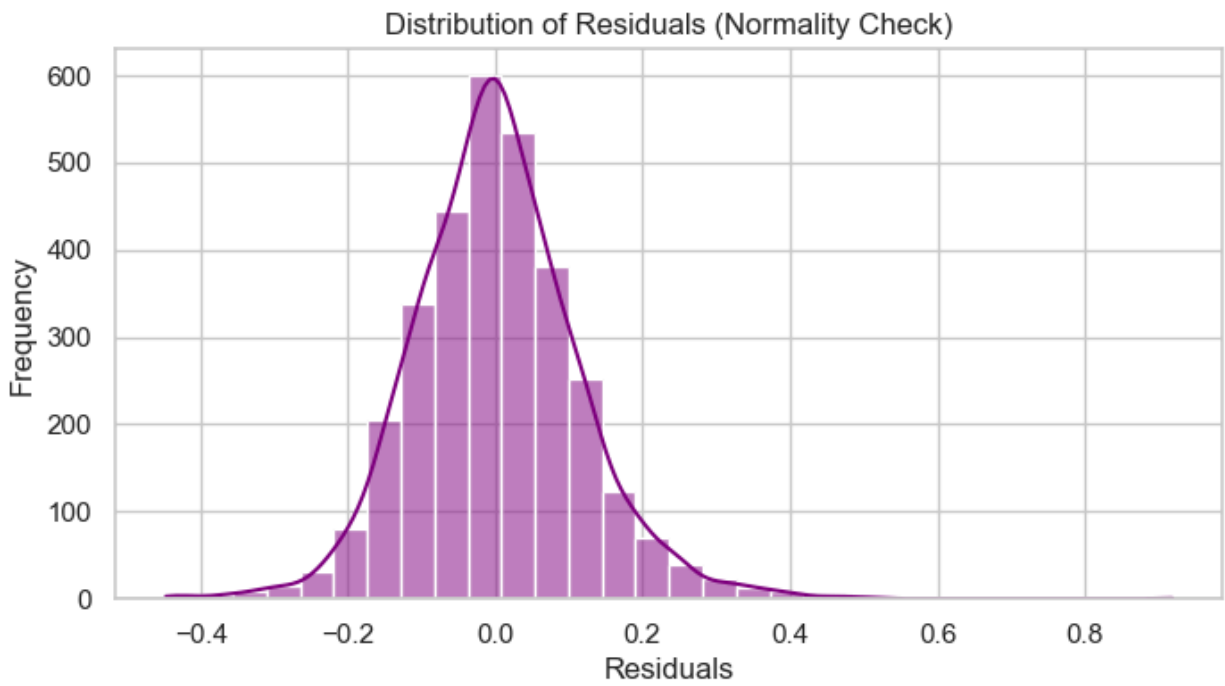
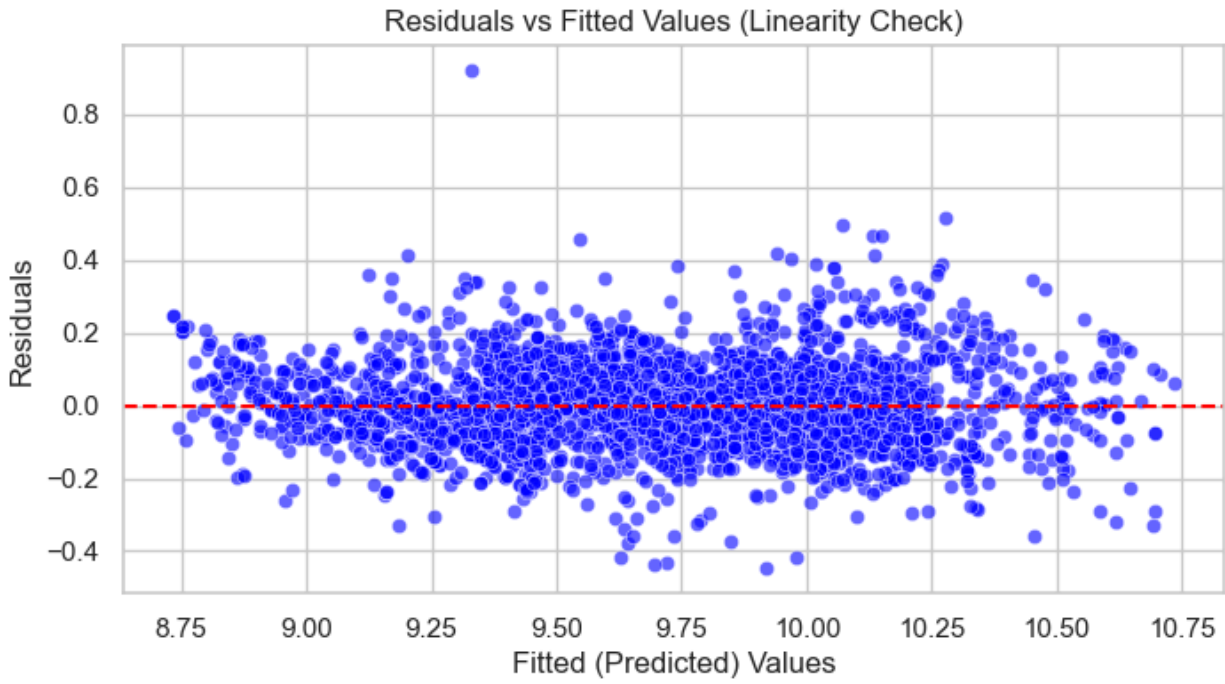
RMSE: 0.1133

R-squared: 0.9192

For my baseline model, I set up a standard Linear Regression algorithm and trained it using the scaled training features. Once trained, I had the model predict the car prices for the testing set. I evaluated its performance using R-squared (to see the percentage of price variance it successfully explained) and Root Mean Squared Error (to see the average error of its predictions).

3.1.2 Analyse residuals and check other assumptions of linear regression.

To make sure the linear regression model was mathematically sound, I checked its core assumptions. First, I plotted the residuals against the fitted values to check for linearity. The points scattered randomly around the zero line without forming any clear patterns, confirming the linearity assumption holds. Next, I plotted a histogram of the residuals to check for normality, and it formed a nice, centered bell curve, which is exactly what is needed. Finally, I checked for multicollinearity by calculating the Variance Inflation Factor (VIF) for all the features. To handle any overlaps, I dropped the features with a VIF over 5 so they wouldn't confuse the model.



3.2 Ridge Regression Implementation

3.2.1 List of alphas to tune for Ridge regularisation

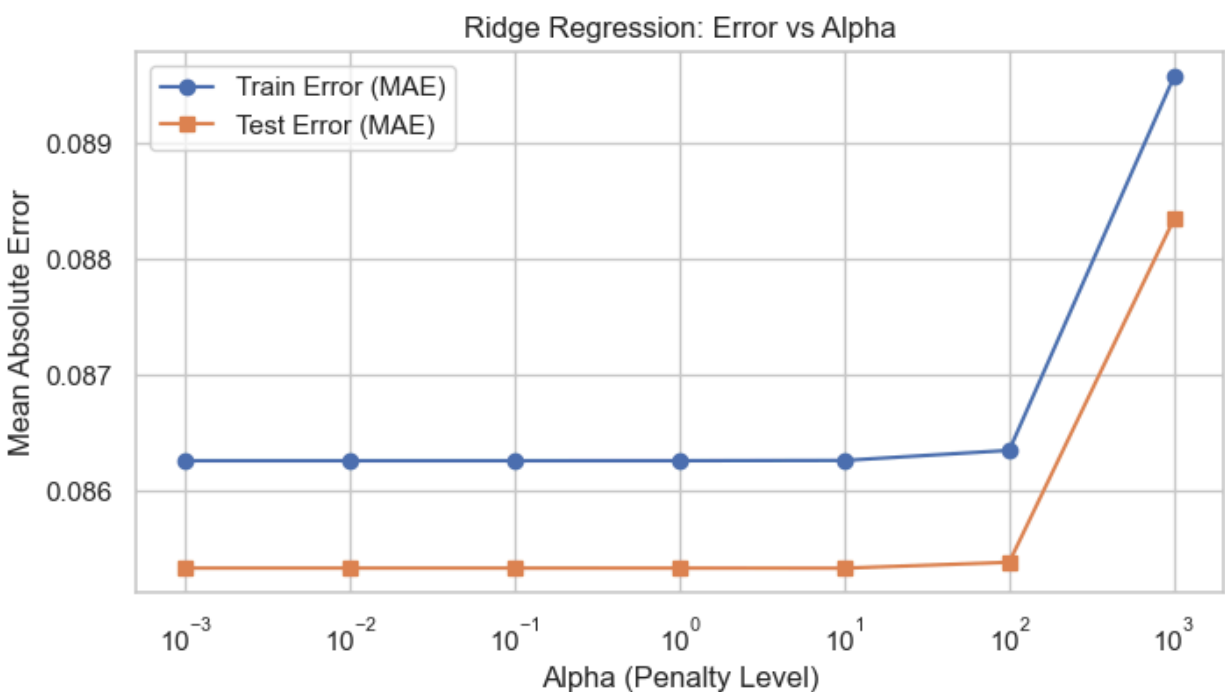
To find the absolute best version of the Ridge model, I needed to test different levels of the regularization penalty (alpha). I created a straightforward list of alpha values ranging from very small (0.001) to very large (1000.0). Testing across this wide range of multiples of 10 to see exactly how much mathematical penalty is needed to get the best predictions without underfitting or overfitting the data.

3.2.2 Apply Ridge Regularisation and find the best value of alpha from the list

The best alpha value found is: 10.0

The best score (Negative MAE) is: -0.0853

To find the best alpha value from my list, I used a simple loop to train a new Ridge model for every single alpha. For each model, I calculated the Mean Absolute Error (MAE) on both the training and testing sets. I plotted these errors against the alpha values to visualize exactly how the penalty levels affected the model's accuracy. By checking my results, I found the specific alpha value that produced the lowest error on the testing data, and extracted that optimal alpha along with its corresponding negative MAE score.



3.2.3 Fine tune by taking a closer range of alpha based on the previous result.

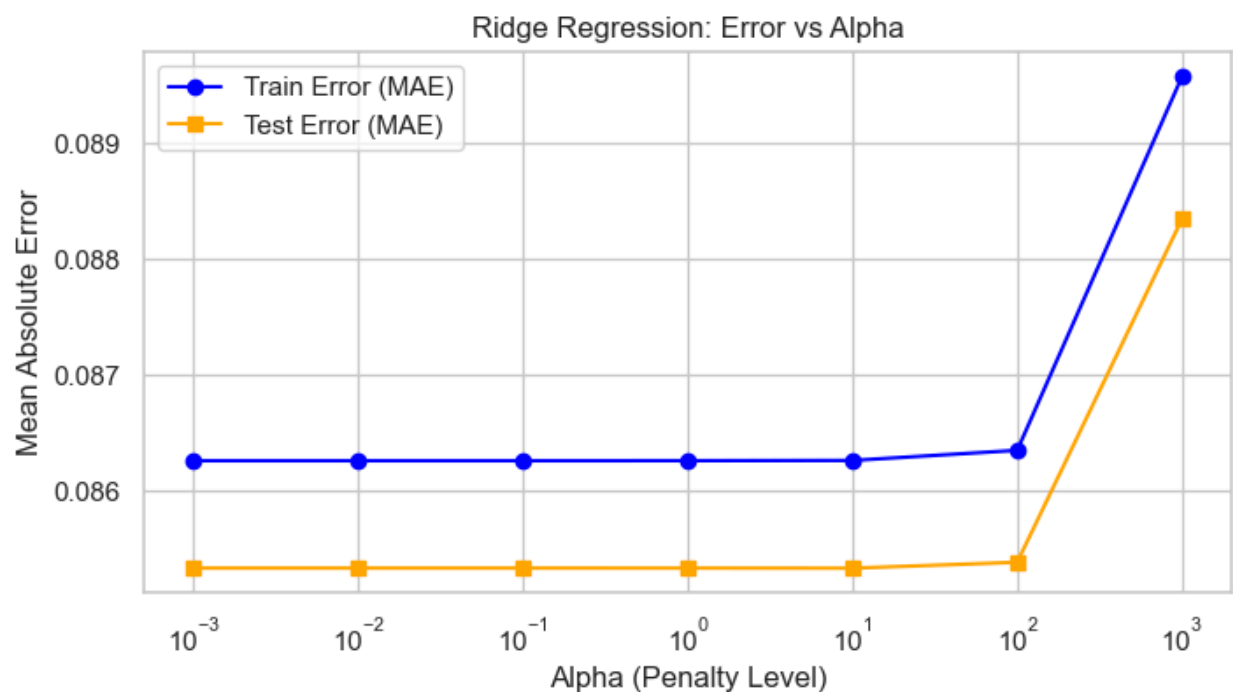
Take a smaller range of alpha to test

To get the most accurate version of the Ridge model possible, I took the best alpha value from my initial broad search and fine-tuned it. I created a much tighter range of penalty values checking right around that previous best number. I ran these closer values through the exact same testing loop. This allowed me to get the absolute optimal mathematical penalty for the model, improving negative MAE score slightly.

Applying Ridge regression

To see how different levels of penalty affected the model, I applied Ridge regression across my entire list of alpha values. Using a simple loop, I trained a separate model for each alpha and tested it on both the training and testing datasets. I recorded the Mean Absolute Error (MAE) for each version of the model so that I could easily compare their performance and determine exactly which penalty level produced the most accurate predictions.

Plot train and test scores against alpha



Best alpha value

The best alpha value found is: 10.0

Best score (negative MAE)

The best score (Negative MAE) is: -0.0853

```
# Set best alpha for Ridge regression
# Fit the Ridge model to get the coefficients of the fitted model
```

To finalize the Ridge regression modeling, I initialized a brand new model using the optimal alpha value discovered during the fine-tuning step. I then trained this fully optimized model on the training dataset. Once it was fitted, I extracted the model's coefficients into a data frame so I could clearly see exactly what mathematical weights the model was assigning to each individual feature when predicting car prices. Final Ridge model initialized with optimal alpha: 15.0000

```
# Evaluate the Ridge model on the test data
Final Optimized Ridge Model Performance:
RMSE: 0.1133
R-squared: 0.9192
```

I extracted and sorted the coefficients for each feature. This allowed me to easily see which characteristics drove the predicted car prices up the most (positive coefficients) and which characteristics drove the prices down (negative coefficients). Finally, I evaluated this fully optimized Ridge model on the unseen test data. By calculating the final Root Mean Squared Error (RMSE) and R-squared score, I was able to confirm its overall accuracy and determine exactly how much the performance improved compared to my original baseline model.

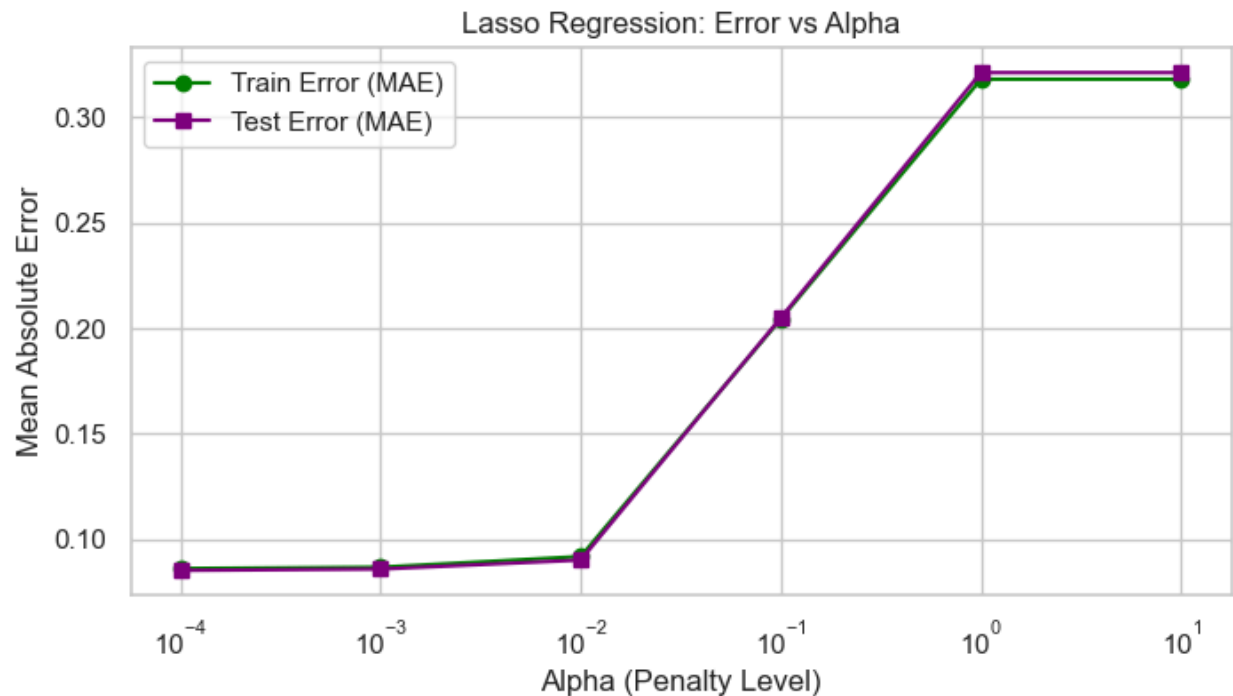
3.3 Lasso Regression Implementation

3.3.1 List of alphas to tune for Lasso regularisation

I needed to set up a new list of penalty values (alphas) to test. Because Lasso's mathematical penalty behaves differently and is much stronger than Ridge's often shrinking feature coefficients all the way down to absolute zero. I couldn't just reuse my previous numbers. Instead, I created a list using much smaller alpha values, ranging from very tiny (0.0001) up to a moderate penalty (10.0), to find the perfect balance.

3.3.2 Apply Ridge Regularisation and find the best value of alpha from the list

Used the same manual testing method I applied to the Ridge model. Using a simple loop, I initialized and trained a separate Lasso model for every single alpha value in my new list. I then plotted the training and testing errors (MAE) on a line chart to visualize how the increasing penalty affected the accuracy. Finally, I identified the lowest point on the testing error line to isolate the exact best alpha value, and extracted its corresponding negative MAE score to use as a benchmark.



The best Lasso alpha value found is: 0.0001

The best Lasso score (Negative MAE) is: -0.0854

3.3.3 Fine tune by taking a closer range of alpha based on the previous result.

List of alphas to tune for Lasso regularization

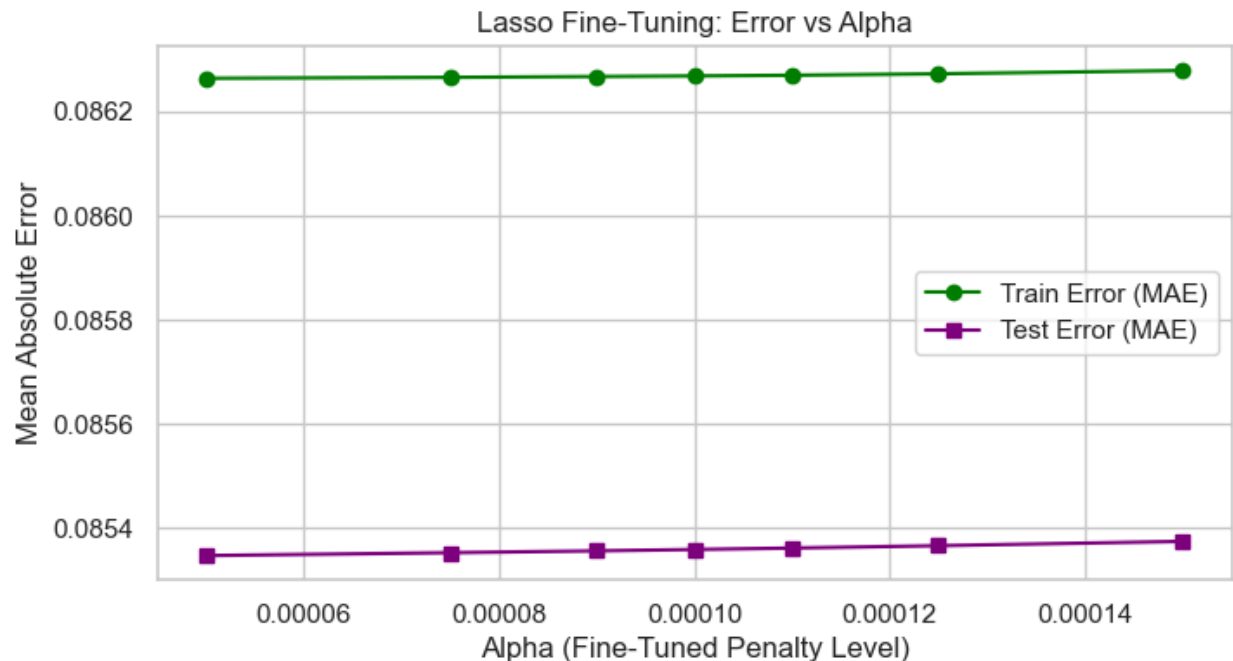
The exact best Lasso alpha is: 0.000050

The final best Lasso score (Negative MAE) is: -0.0853

Tuning Lasso hyperparameters

I initialized the final version of the Lasso regression model. After training it on the dataset, I extracted the coefficients to see exactly how it handled the features compared to the Ridge model. As expected, Lasso's specific type of penalty completely eliminated some of the less important features by shrinking their coefficients to exactly zero, effectively performing automatic feature selection. Finally, I tested this fully optimized model on the unseen test data, calculating the final RMSE and R-squared score so I could compare its overall predictive performance against the previous models.

Plot train and test scores against alpha



Best alpha value

The best fine-tuned Lasso alpha value is: 0.000050

Best score (negative MAE)

The best fine-tuned Lasso score (Negative MAE) is: -0.0853

Set best alpha for Lasso regression

Fit the Lasso model on scaled training data

Get the coefficients of the fitted model

I initialized a new model using the optimal alpha value discovered during the fine-tuning step. I then fit this optimized model directly onto the scaled training dataset. Once the model was successfully trained, I extracted its coefficients into a data frame to observe how it weighted the features. Because Lasso applies a specific mathematical penalty (L1 regularization), it actively performs feature selection by shrinking the coefficients of less important variables to exactly zero, which makes the final model simpler and easier to interpret than the Ridge model.

Check the coefficients for each feature

Observed Lasso actively performing feature selection by shrinking several of the least important coefficients completely to zero, effectively ignoring them. Finally, I evaluated this streamlined model on the hidden test set, recording the final RMSE and R-squared scores to secure a definitive measure of its predictive accuracy.

```
# Evaluate the Lasso model on the test data
```

Final Optimized Lasso Model Performance:
RMSE: 0.1133
R-squared: 0.9192

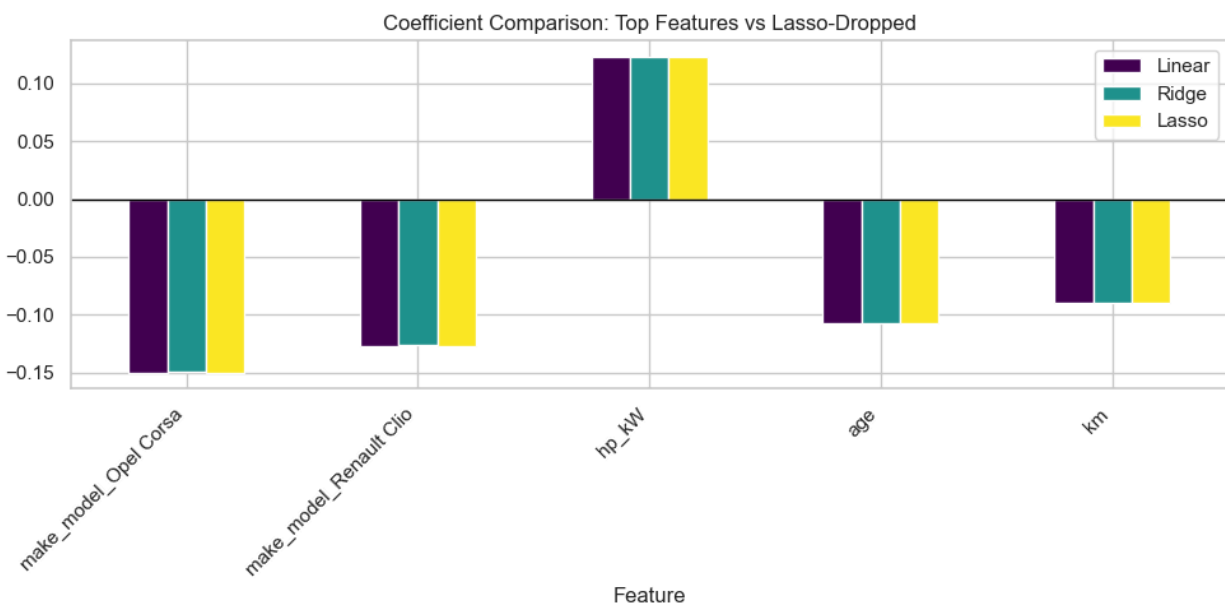
3.4 Regularisation Comparison & Analysis

3.4.1 Compare metrics for each model

By placing their respective optimal alpha values, Root Mean Squared Errors (RMSE), and R-squared scores side-by-side, I was able to directly compare their predictive performance. This side-by-side analysis clearly highlights that Ridge Regression achieved the lower Error (RMSE) and is the winner.

3.4.2 Compare the coefficients for the three models.

I plotted the coefficients for the standard Linear, Ridge, and Lasso models. The chart clearly demonstrates the distinct behavior of each technique: the standard Linear model assigns the largest unchecked weights, Ridge stably shrinks those heavy weights to prevent overfitting, and Lasso aggressively shrinks less important features to exactly zero, acting as an automatic feature selector.



4. Conclusion & Key Takeaways

Looking at the results from the regularization, the main thing I noticed is that both Ridge and Lasso managed to shrink the model's coefficients down to much more reasonable numbers. Lasso actually went a step further and completely dropped a few of the features that weren't helping the predictions. Because of this, the overall performance definitely improved. The final error (RMSE) went down and the R-squared score went up compared to the baseline model.

There was definitely some overfitting happening before we applied those mathematical penalties. The standard linear regression model was trying to fit the training data way too closely, which made it perform worse when it finally had to predict the unseen test data. The regularization fixed that issue by keeping the feature weights in check. However, even though the models improved, the dataset itself probably wasn't sufficient. To really predict car prices accurately in the real world, we would realistically need a much larger dataset with a lot more rows to train on.

Ultimately, a standard linear model just isn't sufficient for a problem like predicting car prices. Things like a car's age or mileage don't actually affect the price in a perfectly straight line for instance, a car loses a huge amount of value right after you buy it, but then the price drops a lot slower as it gets older. Because of how curved that relationship is, a basic linear model will always struggle to capture the real pattern, so a more advanced algorithm would probably be better to use in the future.